

司基于数据采集、数据分析和数据挖掘等技术,帮助企业为客户提供更优良的个性化服务,降低营销成本,提高生产效率,增加利润。大数据分析帮助企业优化管理,调整内部机构,提高服务质量。大数据是未来产业竞争的核心支撑。大数据价值的实现需要通过数据共享、交叉复用才能获得。因此,随着数据成为生产要素,大数据也将会如基础设施一样,有数据提供方、使用方、管理者和监管者等,成为一个大产业。

我国研究者 2012 年就已提出要把数据本身作为研究对象,关注数据科学的研究,研究大数据的科学共性问题<sup>[5]</sup>。数据科学是以大数据为研究对象,横跨信息科学、社会科学、网络科学、系统科学、心理学、经济学等诸多领域的新兴交叉学科。其学科交叉的特点非常鲜明。

大数据可以用于科学研究。2007 年 1 月,James Gray 在加州山景城召开的 NRC-CSTB 上的演讲中提出将人类科学研究分为 4 类范式:以记录和描述自然现象为主的实验科学(第一范式),以模型和归纳为特征的理论科学(第二范式),以模拟仿真为特征的计算科学(第三范式),以及从计算科学中把数据密集型科学区分出来的数据密集型科学发现(第四范式)。James Gray 认为,对于科学研究,科研人员只需从大量数据中查找和挖掘所需要的信息和知识,无须直接面对所研究的物理对象。例如,在天文学领域,天文学家的工作方式发生了大幅度转变。以前,天文学家的主要工作是进行太空拍照。如今,所有照片都已存放在数据库中,天文学家的任务变为从数据库的海量数据中发现有趣的物体或现象。科学研究的第四范式将不仅是研究方式的转变,也是人们思维方式的大变化<sup>[3]</sup>。

关于大数据的特征,IBM 还提出了另一个 V,即 veracity,称为真实性,旨在针对大数据包含的噪声、数据缺失、数据不确定等问题强调数据质量的重要性,以及保证数据质量所面临的巨大挑战。

## 15.2 大数据管理系统

大数据的价值要得到发挥,首要条件是能够被有效管理起来,也就是要解决多类型数据的存取问题:将业务系统产生的具有 4V 特性的大数据及时地存储到系统中,以及高效读取到具有访问权限的业务系统所需要的数据,这就是大数据管理系统的任务。

大数据概念被广泛认可之前,在数据管理的层面上,人们主要关注关系数据库系统,也是本书之前章节主要介绍的内容。也曾有一些观点认为,传统关系数据库能够解决大部分数据管理问题,但这种观点在面临大数据时代的数据管理任务时就已不再成立了。

数据模型是数据管理系统的基础,它与被管理的数据类型密切相关。大数据的多样性带来了不同类型的数据,不同类型的数据有不同的访问接口,导致了不同的数据模型需求。为此,在大数据背景下,人们针对不同类型的数据设计并开发了不同类型的大数据管理系统。如前所述,这些扩展出来的数据类型已经超出了传统数据库所擅长管理的模型数据。本章结合几类最为常见的数据类型:键值对数据、文档数据、图数据及时序数据,分别介绍它们对应的

数据管理系统——大数据管理系统。需要特别说明的是，对于文本、图像、音视频等非结构化数据，我们统一按文档数据类型来抽象，由文档数据库管理。尽管也有一些专门针对多媒体数据管理的数据库，限于篇幅，不对这类数据库展开介绍。

在介绍不同类型的大数据管理系统之前，先简单探讨一下新型大数据管理系统与传统的关系数据库系统的差别，以更好地说明大数据管理系统的特点。

① 大数据管理系统通常是部署在多个节点上的分布式数据管理系统，这是为了能够管理超大规模数据而设计的一类分布式系统。传统的关系数据库虽然也有分布式数据库系统，但主要是单机模式，很少分布在大规模的集群上（近年来吸收了大数据管理系统的特点而有所改进），而大数据管理系统在设计时必须注重系统的可伸缩性（弹性或者可扩展性）。大数据管理系统普遍采用存算分离的松散耦合架构，使得这类系统更容易实现在云上的弹性伸缩。

② 大数据管理系统更强调开源以及采用开放的架构体系。传统关系数据库厂商通常自成体系，因为市场已经成熟甚至形成垄断，所以不愿意走开源开放的技术路线。大数据管理系统则不同，其需求多从互联网公司发起，因此大数据管理系统的设计者更多来自大型互联网公司，他们有高度的开源文化，愿意采用开放的架构来构建系统。这也使得大数据管理系统能够吸引了众多爱好者的参与，在大数据发展的十年间得到了快速发展。

③ 大数据管理系统通常会降低数据质量上的要求。与关系数据不同，大数据通常存在很多数据质量问题，是带有噪声的数据。传统的关系数据库通过数据完整性约束条件的检查与控制机制，在破坏数据库完整性的操作发生时，通过拒绝或报警等方式保护数据库不受侵害。因此，传统的关系数据库的数据质量较高。但是，对于大数据管理系统，数据缺失以及矛盾数据、不完整数据的存在是常态，常需容忍一定的数据质量问题。

### 15.2.1 键值对数据库

键值对数据模型是一种常用的数据模型，指的是每一个数据项由一对信息构成，分别为键和值。键值对数据模型简洁，适应性非常强，可以按照键值对的方式直接使用，也可以组合为更复杂的数据模型。键值对数据库系统由于简单、高效、扩展性强等优点，在很多大数据应用中使用。

#### 1. 键值对数据模型

键值对数据的基本形态是<key,value>，其中key是键，用于标识，一般在系统中不可重复，在键值对数据库中会按照键组织索引，以便加速查找；value是值，值与键一一对应，其信息量大小非常灵活，少则几个字节，多则上千字节，甚至更大。键与值的数据类型也没有限制，但字符串类型居多。

例如，在对学生的基本信息（学号，姓名，性别，出生日期，籍贯，入学成绩）进行管理时，可以使用键值对数据模型，如表15.1所示。

表 15.1 使用键值对数据模型管理学生信息

| key(数字类型) | value(字符串类型)                    |
|-----------|---------------------------------|
| 2021001   | 张三, 男, 2000 年 2 月 20 日, 广西, 668 |
| 2021002   | 李四, 女, 2001 年 5 月 18 日, 广东, 666 |
| 2021003   | 王五, 男, 2000 年 1 月 22 日, 广西, 665 |
| 2021004   | 赵六, 男, 2000 年 8 月 6 日, 广东, 667  |
| .....     | .....                           |

用户通常使用的键值对数据模型的访问接口如表 15.2 所示。

表 15.2 键值对数据模型的访问接口

| 访问接口             | 解释                                                           |
|------------------|--------------------------------------------------------------|
| GET(key)         | 获取 key 对应的值并返回; 或者返回一个值, 表示不存在 key 对应的值                      |
| SET(key, value)  | 写入键值对 <key, value>。有些系统会细分为插入新值的接口 insert 和更新旧值的接口 update 两种 |
| SCAN(key, count) | 范围查询, 从 key 开始, 返回连续 count 个键值对                              |
| SCAN(key1, key2) | 范围查询, 从 key1 开始, 返回 key1 和 key2 之间的所有键值对                     |
| DELETE(key)      | 删除 key 对应的键值对                                                |

此外, 除了采用以上最基本的方式使用键值对数据之外, 还可以用键值对表示其他各种数据模型, 如文档数据模型、关系数据模型、图数据模型等。文档数据模型用键值对表示时, 其中 value 的部分常用 JSON 数据形式(详见 15.2.2 小节)。关系数据模型经常会把一个元组映射为一个键值对, 键由数据库名、表名、元组 ID、时间戳(或版本号)等信息组成, 值包括整个元组各个列的信息。例如, TiDB、TDSQL 等新型分布式数据库就是采用这种映射方法把关系数据映射为键值对数据, 然后存储在底层的分布式键值系统中。图数据映射为键值对数据更为复杂一些, 一般图中的每一个顶点、每一条边都通过一定的规则映射为一个键值对。

## 2. 典型键值对数据库系统

键值对数据库的核心是索引结构, 即如何组织键值对数据。不同索引结构的键值对数据库适用的应用场景不同, 支持的数据操作不同, 系统性能特征也不同。按照常见的索引方式, 键值对数据库可以分为基于哈希索引和基于有序索引两大类型。

哈希索引即利用哈希表作为索引结构, 每个键通过哈希函数的计算, 可以以  $O(1)$  的时间复杂度快速定位到值或其存储位置, 查找和更新都非常迅速。但其缺点是只能支持点查询(如 GET), 不支持范围查询(如 RANGE SCAN)。因此, 如果上层应用不需要支持范围查询, 则基于哈希索引的键值对数据库是很好的选择。例如, Memcached 和 Redis 等开源系统一般用作数据库系统、Web 服务器的缓冲系统, 只需要数据插入、更新和单个键的查询操作, 不需要进行

范围查询。

有序索引指所有键按照一定的顺序排列,这样便于查找,尤其是能够支持快速的范围查询操作。有序索引的类型很多,大部分是各种树形结构,例如  $B$  树、 $B+$  树、前缀树、日志结构合并树(log-structured merge-tree, LSM-tree)等;也有一些是树形结构的变种,如跳表(skip list)。基于有序索引的键值对数据库具有更加丰富的功能,适应更多类型的应用,可以作为数据库的存储层。例如,Facebook 的数据库就构建于自己开发的 MyRocks 存储引擎基础之上,MyRocks<sup>[6]</sup> 的核心结构是基于 LSM-tree 的键值对存储系统,其性能和空间利用率优于传统数据库系统;很多现代分布式数据库产品也都采用 RocksDB(即 MyRocks)作为存储引擎,包括 CockroachDB, TiDB<sup>[7]</sup> 和 TDSQL 等。

$B$  树和  $B+$  树索引是最经典的有序索引结构,能够很好地支持点查询和范围查询,是传统键值对数据库常用的索引结构。但是对于数据主要存储于外存的键值对存储系统, $B/B+$  树索引在更新单个键值对时,很可能需要更新外存中的整个数据块,因此写操作性能较差。目前应用中写操作比例逐渐上升的情况下,基于外存的键值对存储系统逐渐转向采用 LSM-tree 键值对的索引结构。LSM-tree 的数据采用分层结构排列,每一层内的数据都是严格有序的,但层之间允许无序,而且下面的层包含的数据更多。数据写入时会先写入内存中的 MemTable,写满后转为 Immutable MemTable,然后再写入外存,并在后续时间通过后台操作逐渐合并到更下面的层中,如图 15.1 所示。基于 LSM-tree 的键值对数据库的写性能更好,但读性能有所下降,因为要在多个层次中搜索目标数据。典型的基于 LSM-tree 的开源键值对引擎包括 LevelDB 和 RocksDB 等。

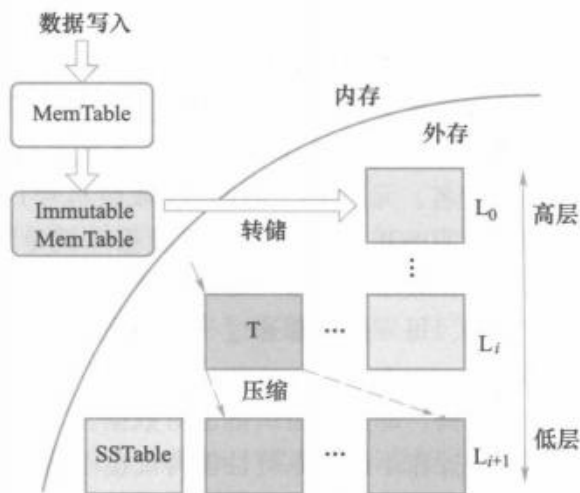


图 15.1 LSM-tree 键值对存储系统的数据组织方式(以 LevelDB 为例)

### 15.2.2 文档数据库

文档数据库用于管理各种各样的文档,主要是文本形式。IBM 的 Lotus Notes 是一款传统的

文档数据库。这里介绍一种 NoSQL 的文档数据库。

### 1. 数据模型和操作

文档数据库一般以键值对数据模型进行数据建模，每个文档是一个<key,value>列表，每个 value 还可以是一个<key,value>列表，形成循环嵌套的结构。由于数据的循环嵌套结构特点，编程负担落在程序员身上，应用程序有可能变得不容易理解和维护。所以，在建模方面需要掌握好灵活性和复杂性之间的平衡。在文档数据库里，可以维护文档的不同历史版本。

在具体实现上，一般采用 JSON(JavaScript object notation)或者类似于 JSON 的格式。JSON 是一种轻量级的、基于文本且独立于语言的数据交换格式，类似于 XML。但是它比 XML 更轻巧，是 XML 数据交换格式的一个替代方案。

这里举一个 JSON 文档格式的例子，假设有两个学生对象，有学号、姓名、性别、出生日期、籍贯、入学成绩、手机号等属性，使用 JSON 进行表示的具体形式如下所示。

```
{
  student:
  |
    sno: "20201121",
    sname: "Wang Ziyi",
    ssex: male,
    sbirthday: 2003-08-15,
    sprovince: "Guang Dong",           /* 籍贯(所在省) */
    sentrance_exam_grade: 688,
    sphone_number: { phone1: 13901117777, phone2: 19301118888 }
  |
}
student:
|
  sno: "20201122",
  sname: "Li Li",
  ssex: female,
  sbirthday: 2003-08-20,
  sprovince: "Guang Xi",
  sentrance_exam_grade: 690,
  sphone_number: { phone1: 13902227777 }
|
}
```

可以看到，JSON 是一种自描述的结构，能够表达信息的层次关系，它给予数据库设计者极大的灵活性以对数据进行建模。

对数据的操作主要包括通过 key 值提取相应的 value, 以及根据 Key 值对 Value 进行修改等。当然, 当 value 本身是一个<key,value>列表时, 需要进一步精细的存取控制。

## 2. MongoDB 数据库

MongoDB 数据库是一款分布式文档数据库, 它针对大数据量、高并发访问性、弱一致性要求的应用而设计。MongoDB 数据库支持极大的扩展能力, 在高负载的情况下, 可以通过添加更多的节点来保证系统的查询性能和吞吐能力。

MongoDB 数据库支持自动分片, 但是数据库的内部架构对应用程序是不可见的。对应用程序而言, 它访问的是一个单机的 MongoDB 数据库服务器还是一个集群, 是没有区别的。

MongoDB 数据库的分片机制可以创建一个包含多个节点的集群, 然后将数据库分片, 分散在集群中, 每个分片维护整个数据集的一个子集。

MongoDB 数据库的分布式体系结构如图 15.2 所示。构建一个 MongoDB 数据库集群需要三个重要的组件, 即分片服务器、配置服务器和路由服务器。

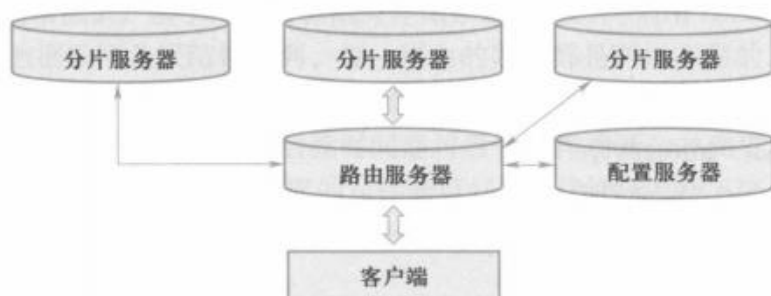


图 15.2 MongoDB 数据库的分布式体系结构

### (1) 分片服务器

每个分片服务器运行一个数据库实例(mongod), 它存储实际的数据块。整个数据集被划分成若干个分片, 每个分片包含若干个数据块, 这些数据块存储在不同的分片服务器中。

在实际应用中, 为了保证系统的可靠性, 一个分片服务器可由几台机器组成一个副本集来承担, 防止因主节点的单点故障而导致整个系统崩溃。

### (2) 配置服务器

配置服务器运行一个独立的 mongod 进程, 它保存整个集群和各个数据分片的元信息, 如各个分片包含了哪些数据等。

### (3) 路由服务器

路由服务器运行一个独立的 mongos 进程。客户端连接到路由服务器, 路由服务器完成路由功能。路由服务器扮演了客户端应用程序和分片集群之间的接口, 使得整个集群看起来是一个单一的数据库。

路由服务器本身不保存数据, 它启动时从配置服务器加载集群的元信息到缓存中, 并将客户端的请求转发给保存数据的分片服务器, 等待各分片服务器返回结果后进行聚合, 然后返回



客户端。

#### (4) MongoDB 数据库的特点和应用

MongoDB 数据库是模式自由的 (schema free)，也就是存储在 MongoDB 数据库中的文件无须事先定义其结构模式。如果有需要，可以把不同结构、不同类型的文档保存到 MongoDB 数据库中。

文档被划分成组，存储在数据库中。一个文档分组称为一个集合 (collection)。每个集合在数据库中有一个唯一的标识，它可以包含无限数目的文档。集合的概念可以对应到关系数据库的表格，而文档则可以对应到关系数据库的一条记录。在这里无须为集合定义任何模式。MongoDB 数据库采用键值对数据模型进行建模，键用于唯一标识一个文档，为字符串类型，而值则可以是任何格式的文件类型。

存储在集合中的文档存储成键值对的形式。MongoDB 数据库还支持二进制的 JSON 形式 (BSON)。BSON 在 JSON 基础上增加了“byte array”数据类型，使得二进制的存储不需要转换成 BASE64 编码后再存为 JSON 格式，大大减少了计算开销以及数据大小。通过 BSON 可以把音频、图像、视频等多媒体数据保存到 MongoDB 数据库中。

MongoDB 数据库支持增加、删除、修改、简单查询等数据操作以及动态查询。可以在复杂属性上建立索引，当查询包含该属性的条件时，可以利用索引加快查询。MongoDB 数据库提供 Ruby、Python、Java、C++、PHP 等语言的编程接口，方便用户使用这些语言编写客户端程序对 MongoDB 数据库进行操作。为了支持业务的持续性，MongoDB 数据库通过复制技术实现节点的故障恢复。

虽然 MongoDB 数据库功能强大，但是它并不能完全替代传统的关系数据库管理系统，因为它缺乏联机事务处理能力。MongoDB 数据库主要应用于大规模、低价值的数据存储和处理场合。比如，欧洲原子能中心使用 MongoDB 数据库来存储大型强子对撞机实验的部分数据。

### 15.2.3 图数据库

图是一种常用的数据结构，由顶点的有穷非空集合以及顶点之间边的集合组成。现实中很多应用场景的数据都可以用图来进行建模，例如社交网络、交通路网、知识图谱、家族图谱等。图数据库除了存储用户数据外，还存储用户之间的各种联系，如好友、同事、同学等联系。

如果图中所有顶点和边各自属于同一类型，则该图称为同构图，否则称为异构图。

图数据库是指面向图数据模型的数据库。需要注意的是，相对于关系模型，图数据模型的说法存在争议，特别是其中的图代数 (对应于关系代数) 以及完整性约束仍然在研究中。进入 21 世纪，随着语义网的发展以及社交网络、知识图谱等大图数据在互联网应用中的普及，应用需求的驱动使得图数据管理的相关研究成为热点。然而，值得探讨的是，与成熟的关系数据库管理系统相比，许多图数据库产品虽然提出了各自的数据模型和查询语言，但仍然缺乏统一的图数据模型和查询语言。为了方便理解，本章引入图数据模型这一概念。

## 1. 图数据结构

目前,图数据库中常用的图数据结构主要包括标签图和属性图两大类。这里介绍属性图。

**定义 15.1** 属性图可表示为一个7元组 $(V, E, L, A, T, \lambda, \sigma)$ , 其中

- ①  $V$  是一组顶点的集合。
- ②  $E$  是一组边的集合。
- ③  $L$  是一组标签的集合。
- ④  $A$  是一组属性的集合。
- ⑤  $T$  是一组属性值的集合。

⑥  $\lambda$  是一个从顶点  $V$ 、边  $E$  到标签  $L$  的映射函数, 即  $\lambda$  函数可以为  $V$  中的每一个顶点和  $E$  中的每一条边产生一个或多个标签。形式化地,  $\forall v \in V, \lambda(v) \subseteq L$  为顶点  $v$  的标签; 类似地,  $\forall e \in E, \lambda(e) \subseteq L$  为边  $e$  的标签。

⑦  $\sigma$  是一个为图中顶点和边产生从属性到属性值的映射函数。形式化地,  $\forall v \in V, \forall a \in A, \exists t \in T, \sigma(v, a) = t$ 。其中, 如果属性  $a$  不被顶点  $v$  所包含, 则属性值  $t$  为空; 否则,  $t$  为顶点  $v$  中属性  $a$  对应的属性值。类似地,  $\forall e \in E, \forall a \in A, \exists t \in T, \sigma(e, a) = t$ 。其中, 如果属性  $a$  不被边  $e$  所包含, 则属性值  $t$  为空, 否则,  $t$  为边  $e$  中属性  $a$  对应的属性值。

图 15.3 给出了学生 Student、课程 Course 及其之间联系的属性图示例。

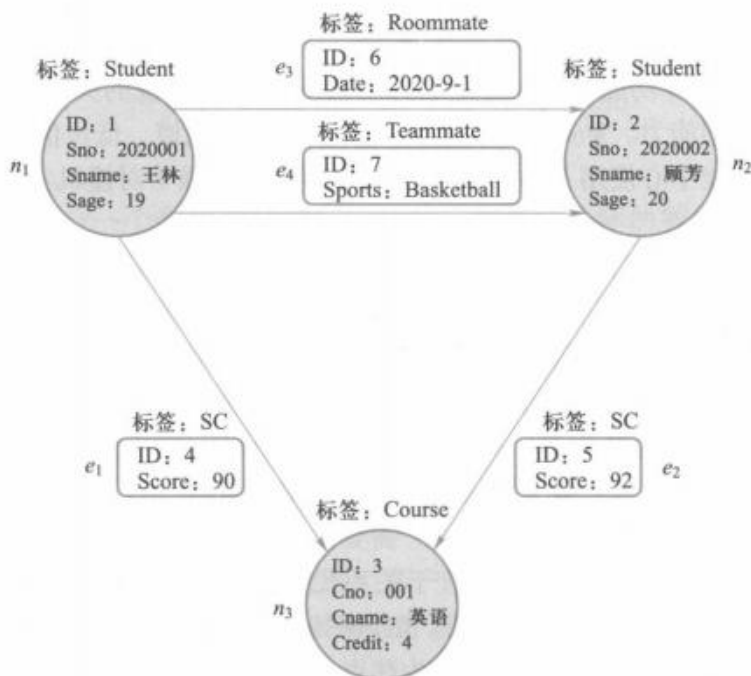


图 15.3 属性图示例



其中,

- ①  $V = \{n_1, n_2, n_3\}$ , 表示图中包含的顶点。
- ②  $E = \{e_1, e_2, e_3, e_4\}$ , 表示图中包含的边。
- ③  $L = \{\text{Student, Course, SC, Roommate, Teammate}\}$ , 表示图中包含的标签。
- ④  $A = \{\text{ID, Sno, Sname, Sage, Cno, Cname, Credit, Score, Date, Sports}\}$ , 表示图中包含的属性。
- ⑤  $T = \{1, 2, 3, 4, 5, 6, 7, 2020001, 2020002, 001, \text{王林, 顾芳, 19, 20, 英语, 90, 92, 4, 2020-9-1, Basketball}\}$ , 表示图中包含的属性值。

⑥  $\lambda(n_1) = \lambda(n_2) = \text{Student}$ ,  $\lambda(n_3) = \text{Course}$ ,  $\lambda(e_1) = \lambda(e_2) = \text{SC}$ ,  $\lambda(e_3) = \text{Roommate}$ ,  $\lambda(e_4) = \text{Teammate}$ , 表示每个顶点或边所对应的标签。例如顶点  $n_1$ 、 $n_2$  的标签为 Student, 对应于关系表 Student 中的两条记录。

⑦  $\sigma(n_1, \text{ID}) = 1$ ,  $\sigma(n_1, \text{Sno}) = 2020001$ ,  $\sigma(n_1, \text{Sname}) = \text{王林}$ ,  $\sigma(n_1, \text{Sage}) = 19$ ;  $\sigma(n_2, \text{ID}) = 2$ ,  $\sigma(n_2, \text{Sno}) = 2020002$ ,  $\sigma(n_2, \text{Sname}) = \text{顾芳}$ ,  $\sigma(n_2, \text{Sage}) = 20$ ;  $\sigma(n_3, \text{ID}) = 3$ ,  $\sigma(n_3, \text{Cno}) = 001$ ,  $\sigma(n_3, \text{Cname}) = \text{英语}$ ,  $\sigma(n_3, \text{Credit}) = 4$ ;  $\sigma(e_1, \text{ID}) = 4$ ,  $\sigma(e_1, \text{Score}) = 90$ ;  $\sigma(e_2, \text{ID}) = 5$ ,  $\sigma(e_2, \text{Score}) = 92$ ;  $\sigma(e_3, \text{ID}) = 6$ ,  $\sigma(e_3, \text{Date}) = 2020-9-1$ ;  $\sigma(e_4, \text{ID}) = 7$ ,  $\sigma(e_4, \text{Sports}) = \text{Basketball}$ 。 $\sigma$  函数描述了每个顶点或每条边所对应的属性及其属性值集合。以顶点  $n_1$  为例,  $\sigma$  函数给出了  $n_1$  包含 4 个属性 ID、Sno、Sname 和 Sage, 对应的属性值分别为 1、2020001、王林和 19。

在属性图中, 标签类似于关系数据库中的关系模式, 标签的实例(顶点或边)类似于关系模式中的记录。顶点或边所包含的 ID 属性实际上是其在系统中的唯一标识, 隐含于创建过程中。为了叙述上的方便, 图 15.3 显式地给出了顶点和边的 ID。

## 2. 图的操作与查询语言

根据对现有主流图数据库、图的应用以及相关文献的调研, 我们把图的操作分为以下三类:

① 图匹配。在图数据中找到满足查询条件的匹配图, 例如针对标签图 RDF 三元组的 SPARQL 查询, 针对标签图、属性图的图同构与子图同构等, 都归属于图匹配操作。

② 图导航。图的路径查询, 包括图中任意两个或多个顶点之间的路径可达查询、最短路径查询、带有限制条件的路径可达查询和最短路径查询等。

③ 图与关系的复合操作。融合关系模型的特有操作(包括投影、选择、连接等)和集合操作(包括并、交、差、笛卡儿积等)到图匹配和图导航中, 增强图操作的表达能力。

结构化查询语言 SQL 是关系数据库的标准查询语言。大部分关系数据库管理系统使用标准 SQL 或其扩展, 这使得 SQL 成为过去数十年用户群中应用最广泛、发展最成熟的数据库查询语言。当借助关系数据库来管理图数据时, 可以使用 SQL 及其扩展语言来实现图的操作。然而, 更普遍的做法是设计专门的图数据库及其类 SQL 查询语言来支持图的操作。专门的图查询语言包括面向标签图查询的 SPARQL、面向属性图查询的 Cypher 和 Gremlin 语言。

## 3. 图数据库 Neo4J

Neo4J 是一个 NoSQL 图数据库管理系统。它采用属性图作为图数据模型, 存储原生的图

数据并提供高效的图算法,能够以相同的速度遍历顶点与边。在关系数据库管理系统中,当使用主码查找某条元组时,通常使用  $B+$  树索引获取该条元组所存储的物理地址,然后根据物理地址读取该条元组;而在 Neo4J 中,由于其特殊的存储结构设计,每个顶点维护其相邻的顶点和边的物理地址,这样在访问邻居时无须通过索引,而是直接使用其物理地址。因此,Neo4J 中顶点遍历速度与图的大小无关,通过某个顶点访问其邻居顶点或边的时间复杂度为  $O(1)$ 。

Neo4J 具有完全的事务管理功能,全面支持 ACID 特性。Neo4J 有两个版本:社区版和商业版。社区版是开源并且免费的,可以支持包含数十亿顶点/边/属性的图,缺点是只能单机使用。商业版与社区版相比,其核心没有太多变化,但支持分布式环境。Neo4J 从 2010 年推出至今被广泛应用于社交网络、推荐引擎、地理数据、物流管理等领域,取得了良好的效果。

Neo4J 简单易用,提供了 API 供 Java、Python、Ruby、PHP、.NET、Node.js 等语言使用。Neo4J 也提供了类似 SQL 的查询语言 CQL(Cypher query language)用于存取数据。实际应用中更多采用 Cypher 语言完成对数据库的增加、删除、修改和查询操作,因为 Cypher 语言的语法更接近于业务逻辑的内涵。常用的 Cypher 语言命令如表 15.3 所示。

表 15.3 常用的 Cypher 语言命令

| 序号 | Cypher 语言命令 | 说明           |
|----|-------------|--------------|
| 1  | CREATE      | 创建节点、联系和属性   |
| 2  | MATCH       | 检索有关节点、联系和属性 |
| 3  | RETURN      | 返回查询结果       |
| 4  | WHERE       | 提供查询条件过滤     |
| 5  | DELETE      | 删除节点和联系      |
| 6  | REMOVE      | 删除节点和联系的属性   |
| 7  | ORDER BY    | 排序检索数据       |
| 8  | SET         | 添加或更新标签      |

以查找学生“王林”的选课信息为例,输出其选修的课程名称及成绩。Cypher 语句如下:

```
MATCH(n:Student)-[e:SC]->(c:Course) WHERE n.Sname='王林'
RETURN c.Cname, e.Score;
```

说明: Cypher 语言使用括号()来表示顶点,使用中括号[]来表示联系,使用大括号{}来表示属性。像 SQL 一样, Cypher 语言提供了 WHERE 子句来进行条件查询,以过滤出通过 MATCH 子句所限定的顶点或联系集合中符合条件的数据。RETURN 子句主要用于返回顶点或联系的属性。

### 15.2.4 时序数据库

时间序列数据是近年来越来越重要的一种数据类型。例如,在云原生应用中,每个微服务占用的 CPU、内存资源及响应请求的耗时等随时间的变化形成多条时间序列数据;在工业物联网应用中,机械设备的发动机转速、发电量、温度等信息随时间的变化也形成多条时间序列。上述这些场景中产生的数据都是用来描述一个对象在时间维度上变化的量,这类数据被称为**时间序列数据**,简称**时序数据**。时序数据具有高通量写入的特性,应用层往往要求数据库具有每秒百万点甚至千万点的写入吞吐能力,管理 TB 级甚至 PB 级的冷/热数据(基于数据访问频度,将数据分为冷数据和热数据),同时还要支持面向时间维度的分组聚合(即多时间分辨率的数据降采样,所谓降采样指的是降低信号采样频率,比如把秒级数据汇总为分钟级数据)等操作,这对传统关系数据库管理系统的性能和查询功能提出了挑战<sup>[8]</sup>。

#### 1. 时序数据模型与操作

一个时序数据库是指多条时间序列形成的集合,其中每条时序可表示为一个序列 ID 和一系列按时间排序的数据点集合:  $\langle \text{tsid}, \{ \langle \text{time}, \text{value} \rangle \} \rangle$ 。在不同实现中,序列 ID(tsId)的表现形式不一,有的以一组<标签,标签值>表示一个序列 ID,有的以一个字符串或长整型表示一个序列 ID;time 一般指时间戳,以毫秒或纳秒为单位;value 可以是数值、布尔值、字符串甚至较大的数组对象,但在现在的时序数据库中,其值仍以数值、布尔值、字符串居多。

以一个新能源汽车管理平台应用为例。在该应用中,每辆新能源汽车是一个产生时序数据的实体,且一辆汽车拥有速度、油耗等多种度量指标,不同车辆可以度量的指标也不尽相同(例如同型号的汽车,某些加装了胎压监测,某些则未安装)。在该应用中,车辆的唯一标识码(即车架号 VIN)与度量指标(如速度、油耗、胎压等)形成唯一的序列 ID。如车架号为 LFV5001 的车的速度序列,车架号为 LFV5001 的车的胎压序列等。在实际应用中,为了便于品牌管理,还可能对汽车实体按品牌分组,形成形如“红旗品牌的车架号为 LFV5001 的车的速度”序列。可以看出,在该应用中,时序数据的模式可被表示为如下所示的树形结构。除省略的序列外,该结构表示了 3 辆汽车上共计 7 条序列 ID。

```
app. Hongqi. LVF5001. speed
app. Hongqi. LVF5001. fuel
app. Hongqi. LVF5001. pressure
app. Hongqi. LVF5002. speed
app. Hongqi. LVF5002. pressure
app. Hongqi. LVF5003. speed
app. Hongqi. LVF5003. pressure
```

进一步,假设速度、油耗需要用浮点数表示,而胎压只需用整数表示,则还需给上述序列增加数据类型定义。在某些时序数据库系统中,序列的元信息不仅包括序列 ID、数据类型,还可能包括静态属性名、标签、频率、编码方法等其他辅助信息。

现有时序数据库由于其数据模型不完全遵从关系模型，因此往往采用类 SQL 或 API 的交互方式。其中，时序数据的典型数据写操作可分为若干类，如表 15.4 所示。

表 15.4 时序数据的典型数据写操作

| 写操作                                                        | 含义                  | 说明                                                  |
|------------------------------------------------------------|---------------------|-----------------------------------------------------|
| <code>insertPoint(entity, metric, time, value)</code>      | 向一条序列写入一个数据点        | entity 如 Hongqi. LVF5001, metric 如速度、胎压，二者共同形成序列 ID |
| <code>insertRecord(entity, metric[], time, value[])</code> | 向同一实体的多个度量写入同一时刻的数据 | 实际应用中，同一实体的多个度量值可能是被同时监测得到的                         |
| <code>insertTablet(entity, metric, time[], value[])</code> | 向一条序列写入一个时间段的数据点    | 实际应用中，同一实体的数据可能是在监测到一段时间的值后批量向数据库写入的                |

上述接口分别覆盖了写一个点、写一行(将一个实体同一时刻的多个度量的值理解为一行数据)和写一(时间)段。在不同的时序数据库系统中可能会对上述写操作接口进行扩展，形成覆盖更丰富的接口。例如，向不同实体的多个度量写入数据。

上述接口的类 SQL 表述一般也比较自然，例如第二个接口在时序数据库管理系统 Apache IoTDB<sup>[9]</sup>中可表示为

```
insert into root. app. Hongqi. LVF5003(time, speed, pressure) values(now(), 1.0, 2);
```

时序数据库的典型查询操作包括：

① 访问某个时刻或某个时间段的一条序列的数据(往往不包含值过滤，但一些时序数据库也支持值过滤)。

② 访问某个时刻或某个时间段的多条序列的数据。

③ 在上述①、②的基础上，不返回原始值，而是返回聚集值(如平均值)。

④ 将一个序列的数据进行降采样：将数据在时间维度上划分成固定长度的时间窗口，在每个窗口内求聚集值，并返回最终结果。

⑤ 将多个序列按某种条件汇聚成一条序列。

其中，查询操作②往往还具有隐式含义——将这些序列的数据按照时间戳进行对齐。不妨用关系数据库来做类比，假设序列 1、序列 2 分别是一个关系表，则所谓的按时间戳进行对齐，即指在查询语句中默认包含了“序列 1 JOIN 序列 2 ON timestamp”。因此，在多数时序数据库管理系统中做多序列同时查询的代价是比较高的(需要做 JOIN 操作)。

查询操作④在许多时序数据库管理系统中采用 GROUP BY 来表示。例如，如果想知道 LVF5003 每小时的平均车速，则查询语句形如：

```
SELECT avg(speed) FROM root. app. Hongqi. LVF5003 GROUP BY(1h);
```

因此, 查询操作④又可称为按时间分组。

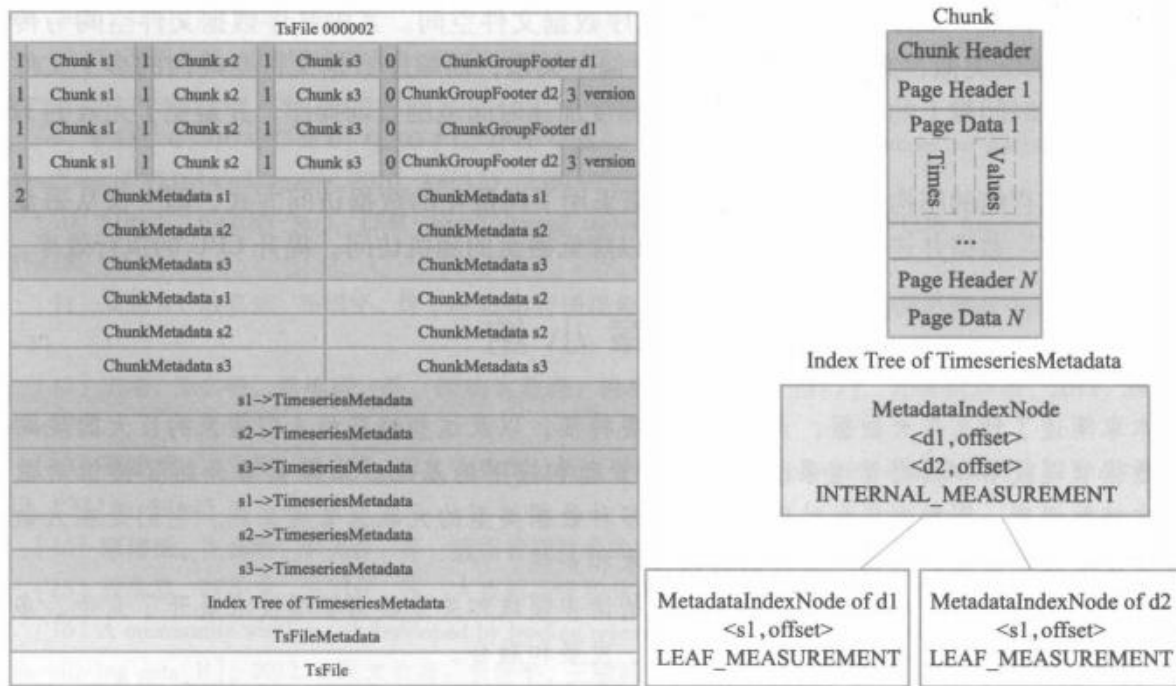
查询操作⑤一般被表述为 GROUP BY TAG/LEVEL，即在用标签描述时序数据模型的系统中称作 GROUP BY TAG，在以树形结构描述时序数据模型的系统中称作 GROUP BY LEVEL。例如，查询红旗汽车的平均油耗的查询语句形如：

```
SELECT avg( fuel) FROM root. app GROUP BY LEVEL=2;
```

## 2. 原生时序数据库管理系统 Apache IoTDB

原生时序数据库管理系统是指那些从数据的物理文件布局到存储、查询引擎都是为时序数据专门研发的系统,其典型代表是 Apache IoTDB<sup>[9]</sup> 和 InfluxDB。

以 Apache IoTDB 为例,不同于传统行式存储,Apache IoTDB 采用了列式文件结构作为数据文件格式,称为 TsFile,用于高效压缩存储以及快速数据访问。TsFile 文件结构如图 15.4 所示。





Header), 用于记录该数据页(块)的统计信息, 如时间范围、值范围等。

在索引区, TsFile 引入了“数据组—数据块”两级多层索引结构(MetadataIndexNode 结构), 替代其他列式文件格式的一维数组查找表结构, 将数据块定位时间复杂度由  $O(n)$  降至  $O(\log n)$ 。

这种存储结构充分考虑了时序数据的读写特点。例如, 由于用户在进行时序数据查询时, 往往同时访问若干条序列进行协同分析, 若这些序列处于同一数据组内, 则无须过多的磁头跳转。又如, 以时序数据管理中重要的降采样查询(例如, 获取某条序列过去 15 分钟内每分钟的平均值)为例, 通过读取数据页头信息中的 SUM 和点数两个统计信息即可计算出该数据页的平均值, 无须读取原始数据。因此, 通过读取若干数据页头信息和少部分原始数据即可得到用户希望的降采样结果。

此外, IoTDB 采用了 NoSQL 数据库常见的日志结构合并(LSM)<sup>[10]</sup>进行多数据文件的整理。但是与传统的 LSM 不同, 由于时序数据整体上服从数据的时间戳不断递增的特点, IoTDB 中的 LSM 由两部分构成: 顺序数据文件空间和乱序数据文件空间。其中乱序数据文件空间与传统 LSM 中的  $L_0$  层级类似, 文件间的数据不存在偏序关系; 而顺序数据文件空间内的多个文件则保持了数据在时间戳上的偏序关系, 从而大幅度加速时间范围查询操作, 并避免了无效的 LSM 写放大。

基于列式存储的结构, IoTDB 的查询引擎采用了向量化的数据访问方式, 即一次从磁盘中读取一个数据块数据并迭代式地遍历数据, 以降低磁盘的随机访问, 提升 CPU 的执行效率。

## 本章小结

本章阐述了什么是大数据, 大数据的重要特征, 以及这些特征给人们带来的巨大的挑战。

数据管理技术和数据管理系统是大数据管理和应用的基础。本章简要介绍了键值对数据库、文档数据库、图数据库和时序数据库等多种数据类型的大数据管理系统。它们是在大数据管理和处理领域涌现的具有代表性的前沿技术和系统。

非关系数据管理技术在数据存储和管理的诸多领域和关系数据管理技术展开了竞争, 多种数据管理系统和相关技术在竞争中相互借鉴、发展和融合。

## 习题 15

1. 什么是大数据, 请简述大数据的基本特征。
2. 请简述键值对数据库的特点(数据模型和主要操作)和典型系统。
3. 请简述文档数据库的特点(数据模型和主要操作)和典型系统。
4. 请简述图数据库的特点(数据模型和主要操作)和典型系统。
5. 请简述时序数据库的特点(数据模型和主要操作)和典型系统。



## 参考文献 15

- [1] LOHR S. The age of big data[N]. The New York Times, 2012-2-11.
- [2] MANYIKA J, CHUI M, BROWN B, et al. Big data: the next frontier for innovation, competition, and productivity[R]. McKinsey Global Institute, 2011.
- [3] 申德荣, 于戈, 王习特, 等. 支持大数据管理的 NoSQL 系统研究综述[J]. 软件学报, 2013, 24(08): 1786-1803.
- [4] PRENTICE S, GENOVESE Y. Pattern-based strategy: getting value from big data[R]. Gartner Research, 2011.
- [5] 李国杰. 大数据研究的科学价值[J]. 中国计算机学会通讯, 2012, 8(9): 8-15.
- [6] MATSUNOBU Y, DONG S Y, LEE H. MyRocks: LSM-Tree database storage engine serving facebook's social graph [C]. Proceedings of the VLDB Endowment, 2020, 13(12): 3217-3230.
- [7] HUANG D X, LIU Q, CUI Q, et al. TiDB: a raft-based HTAP database[C]. Proceedings of VLDB Endowment. 2020, 13(12): 3072-3084.
- [8] JENSEN S K, PEDERSEN T B, THOMSEN C. Time series management systems: a survey[J]. IEEE Transactions on Knowledge and Data Engineering, 2017, 29(11): 2581-2600.
- [9] WANG C, HUANG X D, QIAO J L, et al. Apache IoTDB: time-series database for internet of things[C]. Proceedings of VLDB Endowment, 2020, 13(12): 2901-2904.
- [10] LUO C, CAREY M J. LSM-based storage techniques: a survey[J]. VLDB Journal, 2020, 29(1): 393-418.
- [11] 周晓方, 陆嘉恒, 李翠平, 等. 从数据管理视角看大数据挑战[J]. 计算机学会通讯, 2012, 8(9): 16-20.
- [12] 王珊, 王会举, 覃雄派, 等. 架构大数据: 挑战、现状与展望[J]. 计算机学报, 2011, 34(10): 1741-1752.
- [13] 覃雄派, 王会举, 杜小勇, 等. 大数据分析——RDBMS 与 MapReduce 的竞争与共生[J]. 软件学报, 2012, 23(1): 32-45.
- [14] 覃雄派, 王会举, 李芙蓉, 等. 数据管理技术的新格局[J]. 软件学报, 2013, 24(2): 175-197.
- [15] 程学旗, 靳小龙, 王元卓, 等. 大数据系统和分析技术综述. 软件学报, 2014, 25(9): 1889-1908.
- [16] A community white paper developed by leading researchers across the United States. Challenges and opportunities with big data[R], 2012. (译文节选: 李翠平, 王敏峰. 大数据的挑战和机遇[J]. 科研信息化技术与应用, 2013, 4(1): 12-18)



## 第 16 章 数据仓库与联机分析处理

数据库系统中存在着两类不同的数据处理工作：操作型处理和分析型处理，也称作联机事务处理(OLTP)和联机分析处理(OLAP)。本章首先介绍传统数据仓库与联机分析处理技术，在此基础上介绍大数据时代新型数据仓库与混合事务分析处理(HTAP)技术。

操作型处理也叫事务处理，是指对数据库联机的日常操作，通常是对一个或一组记录的查询和修改，如火车售票系统、银行通存通兑系统、税务征收管理系统等。这些系统要求快速响应用户请求，对数据的安全性、完整性以及事务吞吐量要求很高。

分析型处理是指对数据的查询和分析操作，通常是对海量的历史数据进行查询和分析，如金融风险预测预警系统、证券股市违规分析系统等。这些系统要访问的数据量非常大，查询和分析的操作十分复杂。

OLTP 和 OLAP 两者之间的差异，使得传统的数据库技术不能同时满足两类数据的处理要求。因此，20 世纪 80 年代数据仓库(DW)技术就应运而生了。数据仓库的建立将操作型处理和分析型处理区分开来。传统的数据库系统为操作型处理服务，数据仓库为分析型处理服务，二者各司其职，泾渭分明。越来越多的企业认识到数据仓库能够带来效益，逐步在原有数据库基础之上建立起了自己的数据仓库系统<sup>[1]</sup>。

### 16.1 数据仓库技术

数据仓库和数据库只有一字之差，似乎是一样的概念，但实际则不然。数据仓库是为了构建新的分析处理环境而出现的一种数据存储和组织技术。由于分析处理和事务处理具有极不相同的性质，因而两者对数据也有着不同的要求。W. H. Inmon 在其所著的 *Building the Data Warehouse*<sup>[2]</sup>一书中列出了操作型数据与分析型数据之间的区别，具体如表 16.1 所示。

基于操作型数据和分析型数据之间的区别，Inmon 给出了数据仓库的定义：数据仓库是一个用以更好地支持企业(或组织)决策分析处理的面向主题的、集成的、不可更新的、随时间不断变化的数据集合。数据仓库本质上和数据库一样，是长期储存在计算机内的有组织、可共享的数据集合。